

Mastering Relational and Non-Relational Database

**CHALLENGE
CLYVO VET**

FIAP

Sumario

Folha de rosto ----- 3

Explicação do projeto ----- 4 - 7

DER ----- 8

MER ----- 9

Folha de rosto

Aluno: Lucas Rafael Solimene / RM: 565194

Aluno: Samyr Couto Oliveira / RM: 565562

Aluno: Henrique Teixeira Cesar / RM: 563088

Explicação projeto

Descrição do Projeto – Sistema de Gestão Veterinária

Este projeto consiste no desenvolvimento de um **banco de dados completo para gerenciamento de uma clínica veterinária**, utilizando **Oracle SQL e PL/SQL**. O sistema foi projetado para armazenar, controlar e manipular informações relacionadas a **tutores, pets, consultas, vacinas e histórico de saúde**, garantindo integridade, segurança e rastreabilidade dos dados.

Modelagem do Banco de Dados

O banco foi estruturado com base em um modelo relacional, contendo as seguintes entidades principais:

- **Tutor**: responsável pelo pet
- **Pet**: animal cadastrado no sistema
- **Clínica**: local onde ocorrem as consultas
- **Consulta**: atendimentos realizados
- **Vacina**: tipos de vacinas disponíveis
- **Aplicação de Vacina**: registro das vacinas aplicadas
- **Histórico de Saúde**: registros clínicos dos pets
- **Log de Erros**: armazenamento de falhas do sistema

Relacionamentos

- Um **tutor** pode ter vários **pets**
- Um **pet** pode ter várias **consultas, vacinas e históricos**
- Uma **consulta** ocorre em uma **clínica**
- As tabelas utilizam **chaves primárias (PK)** e **chaves estrangeiras (FK)** para garantir integridade referencial

Procedimentos Armazenados (Procedures)

Foram implementadas procedures para inserção de dados em todas as tabelas principais:

- inserir_tutor
- inserir_pet
- inserir_clinica
- inserir_consulta
- inserir_vacina
- inserir_historico
- inserir_aplicacao_vacina

Funcionalidades das Procedures:

- Realizam inserções de forma controlada
- Tratam exceções com EXCEPTION
- Capturam erros com:
 - SQLCODE
 - SQLERRM
- Registram falhas na tabela **Log_erro**
- Exibem mensagens com DBMS_OUTPUT

Tratamento de Erros

O sistema possui um mecanismo robusto de tratamento de erros:

- Erros como:
 - duplicidade (DUP_VAL_ON_INDEX)
 - violação de chave estrangeira (FK)
 - valores nulos em campos obrigatórios
 - excesso de tamanho de campo
- São capturados nas procedures e registrados na tabela Log_erro

Tabela de Log

Armazena:

- Nome da procedure
- Usuário
- Data do erro
- Código do erro
- Mensagem do erro

Relatórios

Foram implementados relatórios utilizando blocos PL/SQL com FOR LOOP:

1. Consultas por Pet e Tutor

- Mostra quantidade de consultas por pet
- Exibe também o nome do tutor

2. Vacinas por Pet

- Mostra quantas vacinas cada pet recebeu

3. Consultas por Clínica

- Exibe quantas consultas cada clínica realizou

Function

Foi criada a função:

- total_consultas_pet

Objetivo:

Retornar o total de consultas de um pet específico, a partir do seu ID.

Trigger

Trigger implementada:

- validar_idade_pet

Função:

- Impede inserir ou atualizar pets com idade negativa
- Garante validação automática no banco

View

View criada:

- vw_resumo_pet

Informações exibidas:

- Nome do pet
- Nome do tutor
- Total de consultas
- Total de vacinas

Facilita consultas consolidadas e relatórios.

Testes Realizados

Foram executados testes com:

Dados válidos:

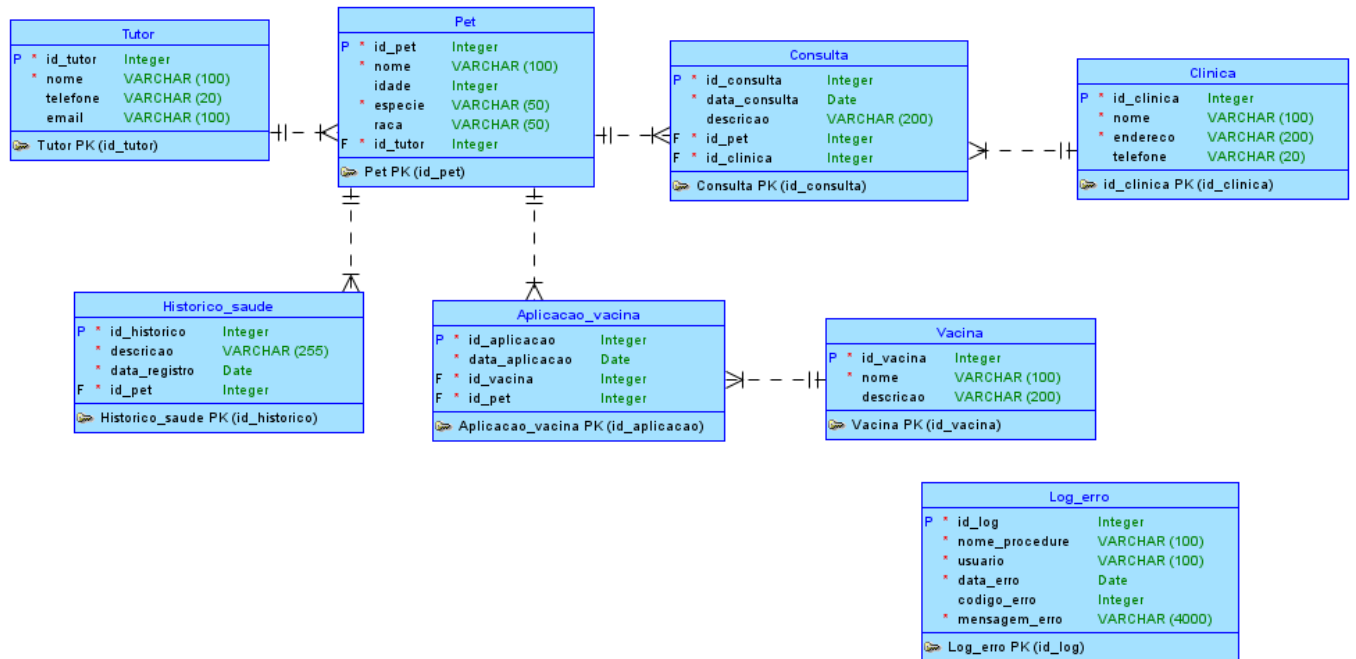
- Inserção completa em todas as tabelas

Dados inválidos:

- Duplicidade
- Chave estrangeira inexistente
- Valores nulos
- Strings maiores que o permitido

Esses testes garantem que o sistema está preparado para lidar com erros reais.

DER



MER

